# From Memorization to Held-Out Validation

Multi-Sentence Training with a Causal PyTorch Attention Model

Week 05 Study Notes — Consciousness Project

November 2026

**Central lesson.** A very small training loss proves that an optimizer fit the training examples. It does not by itself prove that the model learned a rule that works on unseen sequences. Training loss and held-out validation loss answer different questions.

## Contents


**1 Purpose of the experiment** **2**

**2 Vocabulary and shifted next-token supervision** **2**

2.1 Why the encoder handles one sentence . . . . . 3

**3 Causal attention model** **3**

3.1 Causal mask . . . . . 3

3.2 PyTorch implementation . . . . . 4

3.3 Python and PyTorch details . . . . . 4

**4 Cross-entropy and sentence loss** **5**

**5 Epochs and stochastic gradient descent** **5**

**6 Held-out validation** **6**

**7 Observed results** **6**

**8 A subtle limitation of this validation split** **7**

**9 What this experiment proves and does not prove** **7**

**10 Review questions** **8**

**11 Next milestones** **8**

# 1 Purpose of the experiment

The earlier model repeatedly trained on one sentence until it predicted every next token correctly. That experiment verified the complete computational pipeline:

$$\text{text} \to \text{IDs} \to \text{attention} \to \text{logits} \to \text{cross-entropy} \to \text{backpropagation} \to \text{SGD}.$$

However, one sentence cannot distinguish learning from memorization. The next experiment therefore uses several training sequences and one held-out sequence.

The training corpus is

```
I love math !
You love attention !
We love math !
```

and the held-out sequence is

```
We love attention !.
```

All validation tokens occur somewhere in training, but the complete validation sequence is not used for any parameter update.

# 2 Vocabulary and shifted next-token supervision

The shared vocabulary assigns one consecutive integer to every token:

| Token | `I` | `love` | `math` | `We` | `You` | `!` | `attention` |
|---|---|---|---|---|---|---|---|
| ID | 0 | 1 | 2 | 3 | 4 | 5 | 6 |

For a token-ID sequence

$$(t_0, t_1, \ldots, t_m),$$

autoregressive next-token training uses

$$\boldsymbol{x} = (t_0, t_1, \ldots, t_{m-1}), \qquad \boldsymbol{y} = (t_1, t_2, \ldots, t_m).$$

Thus the sentence `I love math !` becomes

$$\boldsymbol{x} = (0, 1, 2), \qquad \boldsymbol{y} = (1, 2, 5).$$

Each input position is paired with the token immediately following it.

```
training_sentences = [
    "I love math !",
    "You love attention !",
    "We love math !",
]
validation_sentence = "We love attention !"

vocabulary = {
    "I": 0, "love": 1, "math": 2, "We": 3,
    "You": 4, "!": 5, "attention": 6,
}

def encode_sentence(sentence, vocabulary):
```

```
    token_ids = []
    for token in sentence.split():
        token_ids.append(vocabulary[token])

    input_ids = torch.tensor(token_ids[:-1], dtype=torch.long)
    target_ids = torch.tensor(token_ids[1:], dtype=torch.long)
    return input_ids, target_ids
```

### 2.1 Why the encoder handles one sentence

The encoding function processes exactly one sequence. The training loop is responsible for calling it once per training sentence. Concatenating all sentences inside the encoder would create false transitions such as

$$\texttt{!} \longrightarrow \texttt{You}$$

between unrelated examples. Keeping these responsibilities separate also makes the function reusable for validation and future datasets.

## 3 Causal attention model

Let $C$ be vocabulary size, $T$ sequence length, $d_{\text{model}}$ embedding width, $d_k$ query/key width, and $d_v$ value width. The trainable layers implement

$$X = E[\boldsymbol{x}] \in \mathbb{R}^{T \times d_{\text{model}}}, \tag{1}$$

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \tag{2}$$

$$S = \frac{QK^\top}{\sqrt{d_k}} + M, \tag{3}$$

$$A = \text{softmax}_{\text{row}}(S), \tag{4}$$

$$O = AV, \tag{5}$$

$$Z = OW_{\text{vocab}}. \tag{6}$$

Here $Z \in \mathbb{R}^{T \times C}$ contains raw vocabulary logits. Row $i$ scores every possible token as the next token after the prefix available at position $i$.

### 3.1 Causal mask

The mask is

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

Because $e^{-\infty} = 0$, position $i$ assigns zero attention probability to future source positions. The row-wise softmax creates a Gibbs distribution over only the permitted sources:

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{r=1}^{T} e^{S_{ir}}}, \qquad \sum_j A_{ij} = 1.$$

### 3.2 PyTorch implementation

```
class ToyAttentionModel(nn.Module):
    def __init__(self, vocab_size, d_model, d_k, d_v):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.query = nn.Linear(d_model, d_k, bias=False)
        self.key = nn.Linear(d_model, d_k, bias=False)
        self.value = nn.Linear(d_model, d_v, bias=False)
        self.output = nn.Linear(d_v, vocab_size, bias=False)

    def forward(self, input_ids):
        X = self.embedding(input_ids)
        Q = self.query(X)
        K = self.key(X)
        V_values = self.value(X)

        d_k = Q.shape[1]
        scores = (Q @ K.T) / (d_k ** 0.5)

        T = input_ids.shape[0]
        mask = torch.full(
            (T, T),
            float("-inf"),
            dtype=X.dtype,
            device=X.device,
        )
        mask = torch.triu(mask, diagonal=1)

        attention = torch.softmax(scores + mask, dim=1)
        O = attention @ V_values
        logits = self.output(O)
        return logits
```

### 3.3 Python and PyTorch details

- `self` is the particular model instance on which a method operates. Calling `model(input_ids)` invokes `model.forward(input_ids)` with `self=model`.
- Components stored as `self.embedding`, `self.query`, and so on persist in the model and are registered by `nn.Module`. Temporary tensors such as $X, Q, K, V, S, A, O, Z$ are recalculated in every forward pass.
- `dtype=X.dtype` makes the mask use the same numerical type as $X$, preventing unintended type conversion during $S + M$.
- `device=X.device` creates the mask on the same CPU, CUDA GPU, or Apple MPS device as $X$. Tensors participating in one operation must normally live on the same device.
- `nn.Linear(a,b,bias=False)` stores a weight of shape $(b, a)$ and evaluates $XW^\top$. This storage orientation differs from the earlier handwritten $XW$ notation, but the transformation is equivalent.

## 4 Cross-entropy and sentence loss

For one sentence with $T$ supervised positions and correct target ID $y_i$ at position $i$, the loss is

$$L_s(\theta) = -\frac{1}{T}\sum_{i=1}^{T} \log \frac{\exp Z_{i,y_i}(\theta)}{\sum_{c=1}^{C} \exp Z_{i,c}(\theta)}. \tag{7}$$

PyTorch evaluates this numerically stable expression with

```
loss = F.cross_entropy(logits, targets)
```

using raw logits. Applying softmax before `F.cross_entropy` would duplicate part of its calculation and reduce numerical stability.

## 5 Epochs and stochastic gradient descent

One *epoch* means visiting every training sentence once. If there are three sentence losses $L_1, L_2, L_3$, the reported epoch loss is

$$\overline{L}_{\text{train}} = \frac{L_1 + L_2 + L_3}{3}.$$

In this experiment the optimizer updates after each sentence, so it performs stochastic gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_\theta L_s(\theta).$$

```
torch.manual_seed(0)
model = ToyAttentionModel(
    vocab_size=len(vocabulary),
    d_model=len(vocabulary),
    d_k=4,
    d_v=5,
)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
training_loss_history = []

for epoch in range(501):
    total_training_loss = 0.0

    for sentence in training_sentences:
        inputs, targets = encode_sentence(sentence, vocabulary)

        optimizer.zero_grad()
        logits = model(inputs)
        loss = F.cross_entropy(logits, targets)
        loss.backward()
        optimizer.step()

        total_training_loss += loss.item()

    average_training_loss = (
        total_training_loss / len(training_sentences)
    )
    training_loss_history.append(average_training_loss)
```

The ordering matters:

1. `zero_grad()` removes derivatives accumulated during the previous update;
2. the forward pass computes predictions using the current parameters;
3. `backward()` applies reverse-mode chain rules and fills parameter gradients;
4. `step()` updates the parameters using those gradients.

The total loss must be accumulated inside the sentence loop, while division and history recording occur after all sentences have been visited. Otherwise, one might accidentally report only the final sentence loss or a cumulative average containing every earlier epoch.

## 6 Held-out validation

Validation asks how the final parameters behave on data that supplied no gradient updates. It uses `model.eval()` and `torch.no_grad()`:

```
model.eval()
validation_inputs, validation_targets = encode_sentence(
    validation_sentence, vocabulary
)

with torch.no_grad():
    validation_logits = model(validation_inputs)
    validation_loss = F.cross_entropy(
        validation_logits, validation_targets
    )
    predicted_ids = torch.argmax(validation_logits, dim=1)
```

`model.eval()` selects evaluation behavior for layers such as dropout and batch normalization. This toy model has neither, so its current numbers do not change, but the call is good general practice. `torch.no_grad()` prevents creation of an unnecessary derivative graph. Validation contains no `backward()` and no optimizer step.

The predicted token at row $i$ is

$$\widehat{y}_i = \arg\max_c Z_{i,c}.$$

A reverse dictionary maps each predicted ID back to readable text.

## 7 Observed results

| Measurement | Value |
|---|---|
| Average training loss after epoch 500 | 0.000868 |
| Held-out validation loss | 2.560667 |
| Validation targets | `love, attention, !` |
| Validation predictions | `love, math, !` |

The first and third predictions generalized correctly:

$$\texttt{We} \longrightarrow \texttt{love}, \qquad \texttt{We love attention} \longrightarrow \texttt{!}.$$

The middle prediction did not:

$$\texttt{We love} \longrightarrow \texttt{math} \quad \text{instead of} \quad \texttt{attention},$$

**Interpretation.** The gap between 0.000868 training loss and 2.560667 validation loss is direct evidence that fitting the three training sentences did not produce reliable behavior on the held-out sequence. The optimizer succeeded at its training objective; the dataset and learned rule did not support the requested generalization.

## 8 A subtle limitation of this validation split

The training set explicitly contains

$$\texttt{We love} \longrightarrow \texttt{math},$$

whereas validation asks the identical prefix to predict

$$\texttt{We love} \longrightarrow \texttt{attention}.$$

The empirical training distribution therefore gives

$$\widehat{P}(\texttt{math} \mid \texttt{We love}) = 1, \qquad \widehat{P}(\texttt{attention} \mid \texttt{We love}) = 0.$$

Driving training cross-entropy toward zero encourages exactly the prediction `math`. Thus this split is useful for exposing memorization and uncertainty, but it is not a clean test in which the held-out answer follows unambiguously from the training labels. Natural language often permits several valid continuations, while the tiny dataset presents each prefix as if it had only one correct continuation.

A stronger future experiment should provide multiple examples supporting a reusable rule, avoid directly contradicting the held-out target for the same prefix, and evaluate several held-out sentences. For example, one could introduce more subjects and objects, reserve combinations whose prefixes were not seen exactly, and report average validation loss across the complete split.

## 9 What this experiment proves and does not prove

It establishes that:

- one model can receive variable sentence examples through a reusable encoder;
- PyTorch correctly accumulates gradients and updates shared embedding and projection weights;
- average epoch loss can be computed across several examples;
- validation can be measured without modifying parameters;
- very low training loss can coexist with very high held-out loss.

It does not establish that:

- the model understands the meanings of `math`, `attention`, or `love`;
- the model generalizes to arbitrary English sentences;
- attention alone is sufficient for a practical language model;
- this one held-out example provides a statistically reliable estimate of performance.

## 10 Review questions

Answer these without running the notebook:

1. Why must token IDs be consecutive integers smaller than vocabulary size?
2. Why does the encoder process one sentence instead of looping over the whole dataset?
3. What false training pairs appear if unrelated sentences are concatenated without a boundary token?
4. What is the difference between a sentence loss and an epoch-average training loss?
5. Why is the epoch loop outside the sentence loop?
6. Why are gradient clearing, backpropagation, and optimizer stepping inside the sentence loop here?
7. Why does validation use neither `backward()` nor `step()`?
8. What do `eval()` and `no_grad()` each do?
9. Why are raw logits passed to cross-entropy?
10. What does the observed train/validation loss gap demonstrate?
11. Why is the chosen validation split partly contradictory?
12. Design a better training/validation split that tests a reusable pattern.

## 11 Next milestones

The most useful progression is:

1. redesign the corpus to give non-contradictory held-out combinations;
2. record both training and validation loss after every epoch and plot them together;
3. add positional encodings and test whether word order changes predictions;
4. introduce mini-batches and padding masks for unequal sequence lengths;
5. compare SGD with Adam and separate learning-rate effects from data effects;
6. train on a larger corpus and report averages across several validation examples.